```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
@Controller
@RequestMapping("/monitor")
public class MonitorController {
    @Autowired
    private DataService dataService;
    @Autowired
    private AlertService alertService;
    //  , ,
    private static final double LOWER_THRESHOLD = 10.0;
    private static final double UPPER_THRESHOLD = 100.0;
    @GetMapping("/checkValues")
    public String checkValues() {
        try {
            // double
            double value = dataService.getValueToMonitor();
            //
            if (value < LOWER_THRESHOLD || value > UPPER_THRESHOLD) {
                //  ,
                String alertMessage = "Value is out of range! Current value: " + value;
                alertService.sendAlert(alertMessage);
                //
                // ...
                //  ,
                return "Error: Value out of range";
            }
            //  ,
            return "Value is within normal range";
        } catch (Exception e) {
            //  ,
            e.printStackTrace(); //  ,
            //  ,
            String alertMessage = "An error occurred during value check. " +
e.getMessage();
            alertService.sendAlert(alertMessage);
            //
            return "Error: " + e.getMessage();
        }
    }
    //  : ,  DataServiceAlertService
    //  ,
}
import java.sql.*;
public class UserDao {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydatabase";
    private static final String USER = "root";
    private static final String PASS = "password";
    //
    public void updateUser(int userId, String name, String email) {
```

```
            String sql = "UPDATE users SET name=?, email=? WHERE id=?";
            try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
                  PreparedStatement pstmt = conn.prepareStatement(sql)) {
                pstmt.setString(1, name);
                pstmt.setString(2, email);
                pstmt.setInt(3, userId);
                int rowsAffected = pstmt.executeUpdate();
                if (rowsAffected == 0) {
                    throw new RuntimeException("No rows updated!");
                }
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
        //  （，）
        public void adjustUserStatus(int userId, boolean isActive) {
            String sql = "UPDATE users SET is_active=? WHERE id=?";
            try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
                  PreparedStatement pstmt = conn.prepareStatement(sql)) {
                pstmt.setBoolean(1, isActive);
                pstmt.setInt(2, userId);
                int rowsAffected = pstmt.executeUpdate();
                if (rowsAffected == 0) {
                    //
                    System.out.println("No rows updated, user may not exist or already in the
desired status.");
                }
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
        //  :，，
        // try-with-resources，。
        //  ，。
}
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.time.Duration;
import java.time.Instant;
@RestController
@RequestMapping("/performance")
public class PerformanceController {
        @Autowired
        private PerformanceService performanceService; //
        @GetMapping("/test")
        public ResponseEntity<PerformanceResult> testPerformance() {
            //
            Instant startTime = Instant.now();
            //  （，）
            String result = performanceService.executeHeavyTask();
```

```
            //
            Instant endTime = Instant.now();
            //
            Duration executionTime = Duration.between(startTime, endTime);
            //
            PerformanceResult performanceResult = new PerformanceResult();
            performanceResult.setResult(result);
            performanceResult.setExecutionTime(executionTime.toMillis()); //
            //
            return ResponseEntity.ok(performanceResult);
        }
        //
        static class PerformanceResult {
            private String result;
            private long executionTime;
            // Getters and setters...
            public String getResult() {
                return result;
            }
            public void setResult(String result) {
                this.result = result;
            }
            public long getExecutionTime() {
                return executionTime;
            }
            public void setExecutionTime(long executionTime) {
                this.executionTime = executionTime;
            }
        }
}
public class ScintillatorMaterialAnalyzer {
        private double decayTime; //
        private double lightOutput; //
        private double density; //
        private double analysisResult; //  （，）
        //  ，（）
        public ScintillatorMaterialAnalyzer() {
            //  ，
            this.decayTime = 1.0;
            this.lightOutput = 100.0;
            this.density = 1.0;
        }
        //
        public void setDecayTime(double decayTime) {
            this.decayTime = decayTime;
        }
        //
        public double getDecayTime() {
            return decayTime;
        }
        //
        public void setLightOutput(double lightOutput) {
            this.lightOutput = lightOutput;
```

```
    }
    //
    public double getLightOutput() {
        return lightOutput;
    }
    //
    public void setDensity(double density) {
        this.density = density;
    }
    //
    public double getDensity() {
        return density;
    }
    //  （）
    public void analyze() {
        //  、（）
        //  ，
        analysisResult = decayTime * lightOutput / density;
        //  ，、
    }
    //
    public double getAnalysisResult() {
        //  ，
        if (analysisResult == 0) {
            throw new IllegalStateException("Analysis has not been performed yet.");
        }
        return analysisResult;
    }
    //  （）
    public static void main(String[] args) {
        ScintillatorMaterialAnalyzer analyzer = new ScintillatorMaterialAnalyzer();
        analyzer.setDecayTime(2.0);
        analyzer.setLightOutput(200.0);
        analyzer.setDensity(0.5);
        analyzer.analyze();
        System.out.println("Analysis result: " + analyzer.getAnalysisResult());
    }
}
import javax.persistence.*;
@Entity
public class LightSource {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    //  （，JSONBBLOB）
    @Column(columnDefinition = "TEXT")
    private String fluctuationData;

    // ...

    // GetterSetter...
```

```
}
import javax.persistence.*;
@Entity
public class ScintillatorResponse {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    //  （）
    @Column(columnDefinition = "TEXT")
    private String responseData;

    //  （）
    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "light_source_id")
    private LightSource lightSource;

    // ...

    // GetterSetter...
}
import org.springframework.data.jpa.repository.JpaRepository;
public interface LightSourceRepository extends JpaRepository<LightSource, Long> {
    //
    //  ：
    // List<LightSource> findByFluctuationData(String fluctuationData);
}
public interface ScintillatorResponseRepository extends JpaRepository<ScintillatorResponse,
Long> {
    //
    //  ：ID
    ScintillatorResponse findByLightSourceId(Long lightSourceId);

    //
    List<ScintillatorResponse> findAllByLightSource(LightSource lightSource);
}
import java.time.LocalDate;
import java.util.HashMap;
import java.util.Map;
public class EquipmentLifecycleManager {
    //  , ,
    private static int predictLifecycle(int age, int conditionFactor) {
        return 100 - (age * 2) - (100 / conditionFactor);
    }
    //
    enum MaintenanceAdvice {
        REPLACE_NOW,
        REPLACE_SOON,
        REGULAR_MAINTENANCE,
        NO_IMMEDIATE_ACTION
    }
    //
    public static MaintenanceAdvice giveMaintenanceAdvice(int age, int conditionFactor) {
```

```
        int predictedLifecycle = predictLifecycle(age, conditionFactor);
        if (age >= predictedLifecycle) {
            return MaintenanceAdvice.REPLACE_NOW; //
        } else if (age >= predictedLifecycle - 10) {
            return MaintenanceAdvice.REPLACE_SOON; //
        } else if (age >= 50) {
            return MaintenanceAdvice.REGULAR_MAINTENANCE; //
        } else {
            return MaintenanceAdvice.NO_IMMEDIATE_ACTION; //
        }
    }
    //  ,
    public static int getCurrentConditionFactor(String equipmentId) {
        //  ,
        //  ,
        return 80; //
    }
    //
    public static Map<String, Object> predictLifecycleAndAdvice(String equipmentId, int
currentAge) {
        int conditionFactor = getCurrentConditionFactor(equipmentId);
        int predictedLifecycle = predictLifecycle(currentAge, conditionFactor);
        MaintenanceAdvice advice = giveMaintenanceAdvice(currentAge, conditionFactor);
        Map<String, Object> result = new HashMap<>();
        result.put("equipmentId", equipmentId);
        result.put("currentAge", currentAge);
        result.put("predictedLifecycle", predictedLifecycle);
        result.put("maintenanceAdvice", advice.name());
        return result;
    }
    //
    public static void main(String[] args) {
        String equipmentId = "EQP-123";
        int currentAge = 45;
        Map<String, Object> result = predictLifecycleAndAdvice(equipmentId, currentAge);
        System.out.println(result);
    }
}
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
public class PredictionDao {
    private static final String JDBC_URL = "jdbc:mysql://localhost:3306/your_database";
    private static final String USER = "your_username";
    private static final String PASSWORD = "your_password";
    // Prediction，
    public static class Prediction {
        private int id;
        private String modelName;
        private double predictionValue;
        private String timestamp;
        // gettersetter
    }
```

```
    public List<Prediction> getPredictionsForReport() {
        List<Prediction> predictions = new ArrayList<>();
        Connection conn = null;
        PreparedStatement pstmt = null;
        ResultSet rs = null;
        try {
            // 1.
            conn = DriverManager.getConnection(JDBC_URL, USER, PASSWORD);
            // 2. PreparedStatement
            String sql = "SELECT id, model_name, prediction_value, timestamp FROM
predictions";
            pstmt = conn.prepareStatement(sql);
            // 3.
            rs = pstmt.executeQuery();
            // 4.
            while (rs.next()) {
                Prediction prediction = new Prediction();
                prediction.setId(rs.getInt("id"));
                prediction.setModelName(rs.getString("model_name"));
                prediction.setPredictionValue(rs.getDouble("prediction_value"));
                prediction.setTimestamp(rs.getString("timestamp"));
                predictions.add(prediction);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // 5.
            try {
                if (rs != null) rs.close();
                if (pstmt != null) pstmt.close();
                if (conn != null) conn.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
        return predictions;
    }
    // ...
}
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.Map;
@RestController
@RequestMapping("/api/datacleansing")
public class DataCleansingController {
    @Autowired
    private DataCleansingService dataCleansingService;
    @PostMapping("/cleanAndDenoise")
    public ResponseEntity<List<Map<String, Object>>>
cleanAndDenoiseData(@RequestBody List<Map<String, Object>> dataList) {
```

```
            try {
                //
                List<Map<String, Object>> cleanedData = dataCleansingService.cleanAndDenoise(dataList);
                //
                return new ResponseEntity<>(cleanedData, HttpStatus.OK);
            } catch (Exception e) {
                //
                e.printStackTrace();
                return new ResponseEntity<>("Error occurred during data cleaning and denoising", HttpStatus.INTERNAL_SERVER_ERROR);
            }
        }
        //  （）
        @PostMapping("/cleanAndDenoiseAsync")
        public ResponseEntity<String> cleanAndDenoiseDataAsync(@RequestBody List<Map<String, Object>> dataList) {
            //
            //  ,
            CompletableFuture.runAsync(() -> {
                try {
                    dataCleansingService.cleanAndDenoiseAsync(dataList);
                    //  , ,
                } catch (Exception e) {
                    //  （, ）
                    e.printStackTrace();
                }
            });
            //  ,
            return new ResponseEntity<>("Data cleaning and denoising task started asynchronously", HttpStatus.ACCEPTED);
        }
}
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
//
class FlashResponseData {
    // ...
    private double intensity; //
    private double frequency; //
    //  、gettersetter...
}
//
interface LightSourceService {
    FlashResponseData analyzeLightSourceFluctuation(double lightIntensity, double lightFrequency);
}
//
@RestController
@RequestMapping("/lightsource")
```

```
public class LightSourceController {
    @Autowired
    private LightSourceService lightSourceService;
    //
    @PostMapping("/analyze")
    public ResponseEntity<FlashResponseData> analyzeLightSource(@RequestBody
LightSourceInput input) {
        try {
            //
            FlashResponseData responseData =
lightSourceService.analyzeLightSourceFluctuation(input.getLightIntensity(),
input.getLightFrequency());
            // ， 200 OK
            return new ResponseEntity<>(responseData, HttpStatus.OK);
        } catch (Exception e) {
            // ， 500 Internal Server Error
            // ，
            e.printStackTrace();
            return new ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }
    //
    static class LightSourceInput {
        private double lightIntensity;
        private double lightFrequency;
        // 、gettersetter...
        public double getLightIntensity() {
            return lightIntensity;
        }
        public void setLightIntensity(double lightIntensity) {
            this.lightIntensity = lightIntensity;
        }
        public double getLightFrequency() {
            return lightFrequency;
        }
        public void setLightFrequency(double lightFrequency) {
            this.lightFrequency = lightFrequency;
        }
    }
}
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
public class DataPersistenceLayer {
    private static final String JDBC_URL = "jdbc:mysql://localhost:3306/mydatabase";
    private static final String USER = "username";
    private static final String PASSWORD = "password";
    // ，
    public static String preprocessData(String data) {
        return data.toUpperCase();
    }
    //
    public static List<String> collectData() {
```

```
        List<String> dataList = new ArrayList<>();
        try (Connection conn = DriverManager.getConnection(JDBC_URL, USER,
PASSWORD);
                Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery("SELECT data_column FROM my_table")) {
            while (rs.next()) {
                String rawData = rs.getString("data_column");
                String preprocessedData = preprocessData(rawData);
                dataList.add(preprocessedData);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return dataList;
    }
    // ， collectData
    public static void main(String[] args) {
        List<String> preprocessedData = collectData();
        for (String data : preprocessedData) {
            System.out.println(data);
        }
    }
    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
import java.util.Random;
public class EnvironmentPredictorService {
    //
    private static final int MIN_TEMPERATURE = 0;
    private static final int MAX_TEMPERATURE = 50;
    //
    private static final int MIN_HUMIDITY = 0;
    private static final int MAX_HUMIDITY = 100;
    //
    private static final String[] PREDICTIONS = {
        "", "", "", "", "", "", ""
    };
    //
    private final Random random = new Random();
    /**
     *
     *
     * @param temperature
     * @param humidity
     * @return
     */
    public String predictEnvironmentImpact(int temperature, int humidity) {
        //
```

```
            if (temperature < MIN_TEMPERATURE || temperature > MAX_TEMPERATURE) {
                throw new IllegalArgumentException("Invalid temperature value");
            }
            if (humidity < MIN_HUMIDITY || humidity > MAX_HUMIDITY) {
                throw new IllegalArgumentException("Invalid humidity value");
            }
            //  ,
            int predictionIndex = 0;
            //
            //  ,
            predictionIndex = temperature / 5 + humidity / 20; //
            //
            predictionIndex  =  Math.max(0,  Math.min(predictionIndex,  PREDICTIONS.length  -
1));
            //
            predictionIndex     =     (predictionIndex     +     random.nextInt(3)     -     1)     %
PREDICTIONS.length;
            //
            return PREDICTIONS[predictionIndex];
        }
        //
        public static void main(String[] args) {
            EnvironmentPredictorService service = new EnvironmentPredictorService();
            int temperature = 25; //
            int humidity = 60; //
            try {
                String prediction = service.predictEnvironmentImpact(temperature, humidity);
                System.out.println("Prediction for temperature=" + temperature + ", humidity=" +
humidity + ": " + prediction);
            } catch (IllegalArgumentException e) {
                System.out.println("Invalid input: " + e.getMessage());
            }
        }
}
public class DataPoint {
        private Long id;
        private String source;
        private Double value;
        private LocalDateTime timestamp;
        //  、gettersetter
        // ...
}
public interface DataPointDAO {
        void insertDataPoint(DataPoint dataPoint);
        List<DataPoint> getDataPointsBySource(String source);
        //  ,
}
import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import javax.transaction.Transactional;
import java.util.List;
@Transactional
public class DataPointDAOImpl implements DataPointDAO {
```

```
    @PersistenceContext
    private EntityManager entityManager;
    @Override
    public void insertDataPoint(DataPoint dataPoint) {
        entityManager.persist(dataPoint);
        //  , 、
    }
    @Override
    public List<DataPoint> getDataPointsBySource(String source) {
        // JPQLCriteria API，JPQL
        String jpql = "SELECT dp FROM DataPoint dp WHERE dp.source = :source";
        TypedQuery<DataPoint> query = entityManager.createQuery(jpql, DataPoint.class);
        query.setParameter("source", source);
        return query.getResultList();
    }
    // ...
    //  （）
    public void batchInsertDataPoints(List<DataPoint> dataPoints) {
        for (DataPoint dataPoint : dataPoints) {
            entityManager.persist(dataPoint);
        }
        // flushclear
    }
}
import java.util.ArrayList;
import java.util.List;
public class PredictionService {
    //
    public static class PredictionData {
        double value;
        String label;
        public PredictionData(double value, String label) {
            this.value = value;
            this.label = label;
        }
        // Getter   Setter ...
    }
    //
    public List<PredictionData> predict(List<Double> inputData) {
        List<PredictionData> predictions = new ArrayList<>();
        for (double input : inputData) {
            //  , ,
            double predictedValue = input * 1.1; //
            String label = "Prediction for " + input;
            predictions.add(new PredictionData(predictedValue, label));
        }
        return predictions;
    }
    //  （）
    public List<String[]> prepareVisualizationData(List<PredictionData> predictions) {
        List<String[]> visualizationData = new ArrayList<>();
        for (PredictionData prediction : predictions) {
            String[] dataRow = new String[2];
```

```
                dataRow[0] = String.valueOf(prediction.getValue()); //
                dataRow[1] = prediction.getLabel(); //
                visualizationData.add(dataRow);
            }
            return visualizationData;
        }
        //  （，）
        public void visualize(List<String[]> visualizationData) {
            for (String[] row : visualizationData) {
                System.out.println("Value: " + row[0] + ", Label: " + row[1]);
                //  ，
                //  ：chart.addDataPoint(Double.parseDouble(row[0]), row[1]);
            }
            // ...
            System.out.println("Prediction visualization completed.");
        }
        //
        public void performPredictionAndVisualization(List<Double> inputData) {
            List<PredictionData> predictions = predict(inputData);
            List<String[]> visualizationData = prepareVisualizationData(predictions);
            visualize(visualizationData);
        }
        //
        public static void main(String[] args) {
            PredictionService service = new PredictionService();
            List<Double> inputData = List.of(10.0, 20.0, 30.0);
            service.performPredictionAndVisualization(inputData);
        }
}
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.List;
//
class PredictionResult {
        //
        String id;
        double value;
        //  、gettersetter
}
//
interface PredictionService {
        List<PredictionResult> predict();
}
//  （）
class MockPredictionService implements PredictionService {
        @Override
        public List<PredictionResult> predict() {
            //
            List<PredictionResult> results = List.of(
                new PredictionResult() {{ id = "1"; value = 0.8; }},
                new PredictionResult() {{ id = "2"; value = 0.95; }}
                //
```

```
            );
            return results;
        }
}
//
class ReportGenerator {
        public void generateReport(List<PredictionResult> results, String filePath) {
            try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath))) {
                //
                writer.write("ID\tValue\n");
                //
                for (PredictionResult result : results) {
                    writer.write(result.id + "\t" + result.value + "\n");
                }
                //
                System.out.println("Report generated successfully and saved to: " + filePath);
            } catch (IOException e) {
                e.printStackTrace();
                System.out.println("Failed to generate report: " + e.getMessage());
            }
        }
}
//
class PredictionReportBusinessLogic {
        private final PredictionService predictionService;
        private final ReportGenerator reportGenerator;
        public PredictionReportBusinessLogic(PredictionService predictionService,
ReportGenerator reportGenerator) {
            this.predictionService = predictionService;
            this.reportGenerator = reportGenerator;
        }
        public void exportAndGenerateReport(String filePath) {
            //
            List<PredictionResult> results = predictionService.predict();
            //
            reportGenerator.generateReport(results, filePath);
        }
        //  ,
        public static void main(String[] args) {
            PredictionService service = new MockPredictionService();
            ReportGenerator generator = new ReportGenerator();
            PredictionReportBusinessLogic logic = new PredictionReportBusinessLogic(service,
generator);
            //
            logic.exportAndGenerateReport("predictions_report.txt");
        }
}
import java.util.concurrent.TimeUnit;
public class PerformanceAssessmentService {
        //  ,  n
        public long calculateFibonacci(int n) {
            if (n <= 1) {
                return n;
```

```
        }
        long a = 0;
        long b = 1;
        for (int i = 2; i <= n; i++) {
            long c = a + b;
            a = b;
            b = c;
        }
        return b;
    }
    //  ,
    public long measurePerformance(int n) {
        long startTime = System.nanoTime();
        long result = calculateFibonacci(n); //
        long endTime = System.nanoTime();
        //  （）
        long elapsedTime = TimeUnit.NANOSECONDS.toMillis(endTime - startTime);
        return elapsedTime;
    }
    //
    //  （100）
    public boolean verifyPerformance(int n, long threshold) {
        long elapsedTime = measurePerformance(n);
        return elapsedTime <= threshold;
    }
    //  ,
    public static void main(String[] args) {
        PerformanceAssessmentService service = new PerformanceAssessmentService();
        // 30，
        long performanceTime = service.measurePerformance(30);
        System.out.println("Performance time for Fibonacci(30): " + performanceTime + "
ms");
        // 100
        boolean isPerformanceAcceptable = service.verifyPerformance(30, 100);
        System.out.println("Is performance acceptable? " + isPerformanceAcceptable);
        // n
        // ...
    }
}
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;
public class ModelUpdaterService {
    //
    private static Map<String, String> modelData = new HashMap<>();
    //
    private static Map<String, String> fetchNewModelData() throws IOException {
        //  ,  API
        //  ,
        Map<String, String> newData = new HashMap<>();
        newData.put("key1", "newValue1");
        newData.put("key2", "newValue2");
```

```
            // ...
            return newData;
        }
        //
        private static boolean shouldUpdateModel() {
            // ，、
            // ， true
            return true;
        }
        //
        public void updateModel() {
            try {
                //
                if (shouldUpdateModel()) {
                    //
                    Map<String, String> newData = fetchNewModelData();
                    // （）
                    synchronized (modelData) {
                        modelData.clear();
                        modelData.putAll(newData);
                    }
                    // ，、
                    System.out.println("Model updated successfully.");
                } else {
                    System.out.println("Model is up to date. No update needed.");
                }
            } catch (IOException e) {
                // ，、
                System.err.println("Error occurred while fetching new model data: " +
e.getMessage());
                // ，，、
            }
        }
        // ：
        public static void main(String[] args) {
            ModelUpdaterService updater = new ModelUpdaterService();
            updater.updateModel();
        }
}
import javax.persistence.*;
@Entity
@Table(name = "equipment")
public class Equipment {
        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Long id;
        @Column(name = "serial_number")
        private String serialNumber;
        @Column(name = "predicted_lifespan")
        private Integer predictedLifespan; // ，
        @Column(name = "maintenance_advice", columnDefinition = "TEXT")
        private String maintenanceAdvice; //
```

```
    // gettersetter...
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getSerialNumber() {
        return serialNumber;
    }
    public void setSerialNumber(String serialNumber) {
        this.serialNumber = serialNumber;
    }
    public Integer getPredictedLifespan() {
        return predictedLifespan;
    }
    public void setPredictedLifespan(Integer predictedLifespan) {
        this.predictedLifespan = predictedLifespan;
    }
    public String getMaintenanceAdvice() {
        return maintenanceAdvice;
    }
    public void setMaintenanceAdvice(String maintenanceAdvice) {
        this.maintenanceAdvice = maintenanceAdvice;
    }
}
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
@Repository
public interface EquipmentRepository extends JpaRepository<Equipment, Long> {
    //
    //
    Equipment findBySerialNumber(String serialNumber);
    //  ，...
    //  ：Spring Data JPA，findBySerialNumber
    // SQLJPQL
}
import java.util.HashMap;
import java.util.Map;
// Service，
public interface UserService {
    String processUserInput(String input);
}
// Service，
public class UserServiceImpl implements UserService {
    @Override
    public String processUserInput(String input) {
        //
        // " processed"
        return input + " processed";
    }
}
//  ，
```

```
public class UserController {
    private final UserService userService;
    public UserController(UserService userService) {
        this.userService = userService;
    }
    //
    public Map<String, Object> handleUserInput(String input) {
        Map<String, Object> response = new HashMap<>();
        //  （）
        if (input == null || input.isEmpty()) {
            response.put("status", "error");
            response.put("message", "Input cannot be empty!");
            return response;
        }
        //
        String processedInput = userService.processUserInput(input);
        //  （）
        long processingTime = System.currentTimeMillis() - /*   */;
        //
        response.put("status", "success");
        response.put("message", processedInput);
        response.put("processingTime", processingTime);
        //  ，、
        // ...
        return response;
    }
    //  ，  handleUserInput
    public static void main(String[] args) {
        UserService userService = new UserServiceImpl();
        UserController controller = new UserController(userService);
        Map<String, Object> response = controller.handleUserInput("Hello, World!");
        System.out.println(response);
    }
}
import java.util.HashMap;
import java.util.Map;
public class PermissionService {
    //
    private static Map<String, String[]> userPermissions = new HashMap<>();
    //
    static {
        userPermissions.put("user1",                new                String[]{"READ_ARTICLES",
"WRITE_COMMENTS"});
        userPermissions.put("user2", new String[]{"READ_ARTICLES"});
        userPermissions.put("admin",                new                String[]{"READ_ARTICLES",
"WRITE_ARTICLES", "DELETE_ARTICLES", "READ_USERS", "WRITE_USERS"});
    }
    //
    public boolean hasPermission(String userId, String permission) {
        String[] userPermissionsArray = userPermissions.get(userId);
        if (userPermissionsArray == null) {
            return false; //
        }
```

```
            for (String p : userPermissionsArray) {
                if (p.equals(permission)) {
                    return true; //
                }
            }
            return false; //
        }
        // ：（READ_ARTICLES）
        public List<Article> getArticleListForUser(String userId) {
            if (!hasPermission(userId, "READ_ARTICLES")) {
                throw new UnauthorizedAccessException("User does not have
READ_ARTICLES permission.");
            }
            //
            List<Article> articles = new ArrayList<>();
            // ...  ...
            return articles;
        }
        // ：（WRITE_COMMENTS）
        public void createComment(String userId, Comment comment) {
            if (!hasPermission(userId, "WRITE_COMMENTS")) {
                throw new UnauthorizedAccessException("User does not have
WRITE_COMMENTS permission.");
            }
            // ... ,  ...
        }
        // ,
        private static class UnauthorizedAccessException extends RuntimeException {
            public UnauthorizedAccessException(String message) {
                super(message);
            }
        }
        // ...  ...
}
// ,
class Article {
        // ...  ...
}
// ,
class Comment {
        // ...  ...
}
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import javax.validation.Valid;
import java.util.Date;
// （DTO）
@RestController
@RequestMapping("/api/realtime")
public class RealtimeDataController {
```

```
    // ，
    @Autowired
    private RealtimeDataService realtimeDataService;
    // POST，
    @PostMapping("/data")
    public ResponseEntity<?> handleRealtimeData(@Valid @RequestBody RealtimeDataDTO dataDTO) {
        try {
            // （，）
            if (dataDTO == null || dataDTO.getValue() == null) {
                return new ResponseEntity<>("Invalid data", HttpStatus.BAD_REQUEST);
            }
            //
            realtimeDataService.processData(dataDTO);
            // （）
            return new ResponseEntity<>("Data received and processed successfully", HttpStatus.OK);
        } catch (Exception e) {
            // （）
            return new ResponseEntity<>(e.getMessage(), HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }
    // （DTO）
    public static class RealtimeDataDTO {
        private String id;
        private Double value;
        private Date timestamp;
        // gettersetter...
        // ...
    }
    //
    public interface RealtimeDataService {
        void processData(RealtimeDataDTO dataDTO);
    }
    // （）
    @Service
    public static class RealtimeDataServiceImpl implements RealtimeDataService {
        @Override
        public void processData(RealtimeDataDTO dataDTO) {
            System.out.println("Received data: " + dataDTO);
            // ，、...
        }
    }
}
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/api/models")
public class ModelUpdateController {
```

```
    // ModelUpdateService
    @Autowired
    private ModelUpdateService modelUpdateService;
    //
    @PostMapping("/auto-update")
    public ResponseEntity<String> autoUpdateModel() {
        try {
            //
            modelUpdateService.autoUpdateModel();
            return new ResponseEntity<>("Model auto-update initiated successfully.",
HttpStatus.OK);
        } catch (Exception e) {
            //
            return new ResponseEntity<>("Failed to initiate model auto-update: " +
e.getMessage(), HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }
    //
    @PutMapping("/adjust-config")
    public ResponseEntity<String> adjustModelConfig(@RequestBody
ModelConfigUpdateRequest request) {
        try {
            //  （）
            //
            modelUpdateService.adjustModelConfig(request.getConfig());
            return new ResponseEntity<>("Model configuration adjusted successfully.",
HttpStatus.OK);
        } catch (Exception e) {
            //
            return new ResponseEntity<>("Failed to adjust model configuration: " +
e.getMessage(), HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }
    // ModelConfigUpdateRequest，
    //  ：，
    static class ModelConfigUpdateRequest {
        private Map<String, Object> config; //
        // GetterSetter...
        public Map<String, Object> getConfig() {
            return config;
        }
        public void setConfig(Map<String, Object> config) {
            this.config = config;
        }
    }
    // ...
    // ModelUpdateService，
    //  ：，
    interface ModelUpdateService {
        void autoUpdateModel() throws Exception;
        void adjustModelConfig(Map<String, Object> config) throws Exception;
        // ...
    }
```

```
    // ... （）
}
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/api/model")
public class ModelController {
    @Autowired
    private ModelService modelService;
    @PostMapping("/train")
    public ResponseEntity<?> trainModel(@RequestBody ModelTrainingParameters
parameters) {
        try {
            //
            boolean trainingResult = modelService.trainModel(parameters);
            if (trainingResult) {
                return new ResponseEntity<>("Model trained successfully",
HttpStatus.OK);
            } else {
                return new ResponseEntity<>("Model training failed",
HttpStatus.INTERNAL_SERVER_ERROR);
            }
        } catch (Exception e) {
            //
            e.printStackTrace();
            return new ResponseEntity<>("An error occurred while training the model",
HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }
    @PostMapping("/optimize")
    public ResponseEntity<?> optimizeModel(@RequestBody
ModelOptimizationParameters parameters) {
        try {
            //
            boolean optimizationResult = modelService.optimizeModel(parameters);
            if (optimizationResult) {
                return new ResponseEntity<>("Model optimized successfully",
HttpStatus.OK);
            } else {
                return new ResponseEntity<>("Model optimization failed",
HttpStatus.INTERNAL_SERVER_ERROR);
            }
        } catch (Exception e) {
            //
            e.printStackTrace();
            return new ResponseEntity<>("An error occurred while optimizing the model",
HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }
    // ModelTrainingParametersModelOptimizationParameters
    //  ,
```

```
    //
}
import java.util.Arrays;
public class PredictionService {
    //  ,
    private final Model1 model1 = new Model1();
    private final Model2 model2 = new Model2();
    private final Model3 model3 = new Model3();
    // 1
    private static class Model1 {
        public double predict(double[] input) {
            //  , ,
            return Math.random(); // 01
        }
    }
    // 2
    private static class Model2 {
        public double predict(double[] input) {
            //  , , ,
            return 1 - Math.random(); // 11
        }
    }
    // 3
    private static class Model3 {
        public double predict(double[] input) {
            //  , ,
            return 0.5 + Math.random() * 0.3; // 0.50.8
        }
    }
    //
    public double ensemblePrediction(double[] input) {
        //
        double prediction1 = model1.predict(input);
        double prediction2 = model2.predict(input);
        double prediction3 = model3.predict(input);
        //  ,
        double ensemblePrediction = (prediction1 + prediction2 + prediction3) / 3;
        //  , 、
        //  ,
        return ensemblePrediction;
    }
    //  ,
    public static void main(String[] args) {
        PredictionService service = new PredictionService();
        double[] input = {1.0, 2.0, 3.0}; //
        double prediction = service.ensemblePrediction(input);
        System.out.println("Ensemble Prediction: " + prediction);
    }
}
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
```

```
import org.springframework.web.bind.annotation.*;
import java.util.List;
//
interface DataProcessingService {
    List<String> collectData();
    List<String> preprocessData(List<String> data);
}
//
class MockDataProcessingService implements DataProcessingService {
    @Override
    public List<String> collectData() {
        //  , ,
        return List.of("Raw Data 1", "Raw Data 2", "Raw Data 3");
    }
    @Override
    public List<String> preprocessData(List<String> data) {
        //  , ,（）
        return data; //  ,
    }
}
//
@RestController
@RequestMapping("/data")
public class DataController {
    @Autowired
    private DataProcessingService dataProcessingService;
    @GetMapping("/collect")
    public ResponseEntity<List<String>> collectData() {
        try {
            List<String> rawData = dataProcessingService.collectData();
            if (rawData == null || rawData.isEmpty()) {
                return new ResponseEntity<>("No data found", HttpStatus.NOT_FOUND);
            }
            return new ResponseEntity<>(rawData, HttpStatus.OK);
        } catch (Exception e) {
            e.printStackTrace();
            return new ResponseEntity<>("Error occurred during data collection",
HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }
    @PostMapping("/preprocess")
    public ResponseEntity<List<String>> preprocessData(@RequestBody List<String>
rawData) {
        try {
            if (rawData == null || rawData.isEmpty()) {
                return new ResponseEntity<>("No data provided for preprocessing",
HttpStatus.BAD_REQUEST);
            }
            List<String> preprocessedData =
dataProcessingService.preprocessData(rawData);
            return new ResponseEntity<>(preprocessedData, HttpStatus.OK);
        } catch (Exception e) {
            e.printStackTrace();
```

```
                return new ResponseEntity<>("Error occurred during data preprocessing",
HttpStatus.INTERNAL_SERVER_ERROR);
            }
        }
}
import java.sql.*;
public class FlashingMaterialDao {
    private static final String JDBC_URL = "jdbc:mysql://localhost:3306/your_database";
    private static final String USER = "your_username";
    private static final String PASSWORD = "your_password";
    private Connection getConnection() throws SQLException {
        return DriverManager.getConnection(JDBC_URL, USER, PASSWORD);
    }
    public void insertFlashingMaterial(String name, double density, double efficiency) throws
SQLException {
        String sql = "INSERT INTO FlashingMaterial (name, density, efficiency) VALUES
(?, ?, ?)";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, name);
            pstmt.setDouble(2, density);
            pstmt.setDouble(3, efficiency);
            pstmt.executeUpdate();
        }
    }
    public FlashingMaterial getFlashingMaterialById(int id) throws SQLException {
        String sql = "SELECT * FROM FlashingMaterial WHERE id = ?";
        FlashingMaterial material = null;
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, id);
            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                material = new FlashingMaterial(rs.getInt("id"), rs.getString("name"),
rs.getDouble("density"), rs.getDouble("efficiency"));
            }
        }
        return material;
    }
    public void updateFlashingMaterial(int id, double newDensity, double newEfficiency)
throws SQLException {
        String sql = "UPDATE FlashingMaterial SET density = ?, efficiency = ? WHERE id
= ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setDouble(1, newDensity);
            pstmt.setDouble(2, newEfficiency);
            pstmt.setInt(3, id);
            pstmt.executeUpdate();
        }
    }
    public void deleteFlashingMaterial(int id) throws SQLException {
        String sql = "DELETE FROM FlashingMaterial WHERE id = ?";
```

```
            try (Connection conn = getConnection();
                  PreparedStatement pstmt = conn.prepareStatement(sql)) {
                pstmt.setInt(1, id);
                pstmt.executeUpdate();
            }
        }
        // FlashingMaterial （）
        static class FlashingMaterial {
            int id;
            String name;
            double density;
            double efficiency;
            // 、getter  setter
        }
}
public class ScintillatorAnalyzerController {
        private double lightIntensity; //
        private double decayTime; //
        private String spectrumDistribution; //  （）
        // ，（）
        public ScintillatorAnalyzerController() {
            this.lightIntensity = 0.0;
            this.decayTime = 0.0;
            this.spectrumDistribution = "Unknown";
        }
        //
        public void setLightIntensity(double lightIntensity) {
            if (lightIntensity >= 0) {
                this.lightIntensity = lightIntensity;
            } else {
                System.out.println("，。");
            }
        }
        //
        public double getLightIntensity() {
            return lightIntensity;
        }
        //
        public void setDecayTime(double decayTime) {
            if (decayTime >= 0) {
                this.decayTime = decayTime;
            } else {
                System.out.println("，。");
            }
        }
        //
        public double getDecayTime() {
            return decayTime;
        }
        //  （，）
        public void setSpectrumDistribution(String spectrumDistribution) {
            this.spectrumDistribution = spectrumDistribution;
```

```
    }
    //
    public String getSpectrumDistribution() {
        return spectrumDistribution;
    }
    //  （，）
    public void analyzeScintillatorParameters() {
        System.out.println("：");
        System.out.println(": " + lightIntensity);
        System.out.println(": " + decayTime + " ");
        System.out.println(": " + spectrumDistribution);
    }
    //  （）
    public void simulateAndAnalyze() {
        //
        double simulatedLightIntensity = 100.0;
        double simulatedDecayTime = 0.5;
        String simulatedSpectrumDistribution = "Visible Light";
        //
        setLightIntensity(simulatedLightIntensity);
        setDecayTime(simulatedDecayTime);
        setSpectrumDistribution(simulatedSpectrumDistribution);
        //
        analyzeScintillatorParameters();
    }
    //  ，（）
    public static void main(String[] args) {
        ScintillatorAnalyzerController controller = new ScintillatorAnalyzerController();
        controller.simulateAndAnalyze();
    }
}
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/features")
public class FeatureSelectionController {
    @Autowired
    private FeatureSelectionService featureSelectionService;
    // POST，
    @PostMapping("/select")
    public ResponseEntity<String> selectFeatures(@RequestBody List<String>
inputFeatures) {
        try {
            //
            List<String> selectedFeatures =
featureSelectionService.selectFeatures(inputFeatures);

            //  ,
            //  ,
```

```
                if (selectedFeatures != null && !selectedFeatures.isEmpty()) {
                    //
                    return new ResponseEntity<>("Selected features: " + String.join(", ",
selectedFeatures), HttpStatus.OK);
                } else {
                    // ,
                    return new ResponseEntity<>("No features selected.",
HttpStatus.BAD_REQUEST);
                }
            } catch (Exception e) {
                // ,
                return new ResponseEntity<>("Error occurred during feature selection: " +
e.getMessage(), HttpStatus.INTERNAL_SERVER_ERROR);
            }
        }
        // GET，
        @GetMapping("/extract")
        public ResponseEntity<List<String>> extractFeatures(@RequestParam String dataId) {
            try {
                //
                List<String> extractedFeatures =
featureSelectionService.extractFeatures(dataId);

                // ,
                // ,

                if (extractedFeatures != null && !extractedFeatures.isEmpty()) {
                    //
                    return new ResponseEntity<>(extractedFeatures, HttpStatus.OK);
                } else {
                    // ,
                    return new ResponseEntity<>(HttpStatus.NOT_FOUND);
                }
            } catch (Exception e) {
                // ,
                return new ResponseEntity<>("Error occurred during feature extraction: " +
e.getMessage(), HttpStatus.INTERNAL_SERVER_ERROR);
            }
        }
}
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
//   Feature ,
class Feature {
    String name;
    double score;
    Feature(String name, double score) {
        this.name = name;
        this.score = score;
```

```
    }
    // Getter    Setter
    // ...
}
//
class FeatureSelectionService {
    //  （）
    private Map<String, Double> evaluateFeatures(List<String> featureNames) {
        //  ，
        Map<String, Double> scores = Map.of(
            "feature1", 0.8,
            "feature2", 0.5,
            "feature3", 0.9,
            "feature4", 0.2,
            // ...
        );
        return scores;
    }
    //  （N）
    public List<Feature> selectTopFeatures(List<String> featureNames, int topN) {
        //
        Map<String, Double> scores = evaluateFeatures(featureNames);
        // Feature
        List<Feature> features = new ArrayList<>();
        for (Map.Entry<String, Double> entry : scores.entrySet()) {
            features.add(new Feature(entry.getKey(), entry.getValue()));
        }
        //
        features.sort(Comparator.comparingDouble(Feature::getScore).reversed());
        // N
        if (topN > features.size()) {
            topN = features.size();
        }
        return features.subList(0, topN);
    }
    //  ，
    public void displaySelectedFeatures(List<Feature> selectedFeatures) {
        for (Feature feature : selectedFeatures) {
            System.out.println("Feature    Name:    "    +    feature.name    +    ",    Score:    "    +
feature.score);
        }
    }
}
//
public class Main {
    public static void main(String[] args) {
        FeatureSelectionService service = new FeatureSelectionService();
        //
        List<String>  featureNames  =  List.of("feature1",  "feature2",  "feature3",  "feature4",
"feature5");
        // 3
        int topN = 3;
        List<Feature> selectedFeatures = service.selectTopFeatures(featureNames, topN);
```

```
            //
            service.displaySelectedFeatures(selectedFeatures);
        }
}
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.MediaType;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;
import java.util.Map;
@RestController
@RequestMapping("/predictions")
public class PredictionController {
    @Autowired
    private PredictionService predictionService;
    //  （）
    private static class VisualizationTool {
        //  ，
        //  , ,
        public static String visualize(List<Map<String, Object>> predictions) {
            // JSON
            //  ，SVG，
            StringBuilder sb = new StringBuilder("[");
            for (int i = 0; i < predictions.size(); i++) {
                if (i > 0) {
                    sb.append(",");
                }
                sb.append(predictions.get(i).toString());
            }
            sb.append("]");
            return sb.toString();
        }
    }
    @GetMapping(value = "/get", produces = MediaType.APPLICATION_JSON_VALUE)
    public String getPredictions() {
        //
        List<Map<String, Object>> predictions = predictionService.getPredictions();
        //
        //  :，URL、
        String visualizedData = VisualizationTool.visualize(predictions);
        //  （，HTTP，、JSON）
        return visualizedData;
    }
    //  ：PredictionService
    // ...
}
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
```

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {
    @Autowired
    private AuthService authService;
    //
    @GetMapping("/permissions")
    public ResponseEntity<List<String>> getPermissions(@RequestHeader("Authorization")
String token) {
        try {
            // token
            // ：authService.validateToken(token);
            List<String> permissions = authService.getUserPermissions(token);
            return new ResponseEntity<>(permissions, HttpStatus.OK);
        } catch (Exception e) {
            // ，token、
            return new ResponseEntity<>("Invalid token or user not found",
HttpStatus.UNAUTHORIZED);
        }
    }
    // （）
    @PutMapping("/permissions/{userId}")
    public ResponseEntity<Void> updatePermissions(@PathVariable("userId") String userId,
                                                  @RequestBody List<String>
newPermissions,
                                                  @RequestHeader("Authorization")
String adminToken) {
        try {
            // token
            // ：authService.validateAdminToken(adminToken);
            //
            authService.updateUserPermissions(userId, newPermissions);
            return new ResponseEntity<>(HttpStatus.OK);
        } catch (Exception e) {
            // ，token、、
            return new ResponseEntity<>("Invalid token, user not found, or invalid
permissions", HttpStatus.BAD_REQUEST);
        }
    }
    //
    @GetMapping("/check-permission/{userId}/{permission}")
    public ResponseEntity<Boolean> checkPermission(@PathVariable("userId") String
userId,
                                                   @PathVariable("permission")
String permission,
                                                   @RequestHeader("Authorization")
String token) {
        try {
            // token
            // ：authService.validateToken(token);
            Boolean hasPermission = authService.checkUserPermission(userId,
permission);
```

```
                return new ResponseEntity<>(hasPermission, HttpStatus.OK);
            } catch (Exception e) {
                // ，token、
                return new ResponseEntity<>("Invalid token or user not found",
HttpStatus.UNAUTHORIZED);
            }
        }
        // ...
}
import java.util.HashMap;
import java.util.Map;
// A
class ModelA {
        public ModelA() {
            // ModelA
            System.out.println("ModelA is initialized.");
        }
        // ModelA...
}
// B
class ModelB {
        public ModelB() {
            // ModelB
            System.out.println("ModelB is initialized.");
        }
        // ModelB...
}
//
public class ModelController {
        // ，
        private static final Map<String, Class<?>> MODEL_FACTORY = new HashMap<>();
        // ，
        static {
            MODEL_FACTORY.put("ModelA", ModelA.class);
            MODEL_FACTORY.put("ModelB", ModelB.class);
        }
        //
        public Object initializeModel(String modelName) throws InstantiationException,
IllegalAccessException {
            //
            if (!MODEL_FACTORY.containsKey(modelName)) {
                throw new IllegalArgumentException("Invalid model name: " + modelName);
            }
            //
            Class<?> modelClass = MODEL_FACTORY.get(modelName);
            //
            return modelClass.getDeclaredConstructor().newInstance();
        }
        // ：
        public Object autoSelectAndInitializeModel(SomeCondition condition) {
            String modelName;
            //
            if (condition.someConditionMet()) {
```

```
                modelName = "ModelA";
            } else {
                modelName = "ModelB";
            }
            try {
                return initializeModel(modelName);
            } catch (Exception e) {
                //
                throw new RuntimeException("Failed to initialize model", e);
            }
        }
        // SomeCondition，
        interface SomeCondition {
            boolean someConditionMet();
        }
        //  ,
        public static void main(String[] args) {
            ModelController controller = new ModelController();
            try {
                Object modelA = controller.initializeModel("ModelA");
                // modelA...
                Object modelB = controller.initializeModel("ModelB");
                // modelB...
                //
                SomeCondition condition = () -> true; // true，
                Object autoSelectedModel = controller.autoSelectAndInitializeModel(condition);
                // autoSelectedModel...
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
}
import java.util.ArrayList;
import java.util.List;
public class DataCleaningService {
        //
        private static final List<String> NOISE_STRINGS = new ArrayList<>();
        static {
            NOISE_STRINGS.add("noise1");
            NOISE_STRINGS.add("noise2");
            //
        }
        /**
         *
         *
         * @param dataList
         * @return
         */
        public List<String> cleanAndDenoiseData(List<String> dataList) {
            if (dataList == null || dataList.isEmpty()) {
                return new ArrayList<>();
            }
            List<String> cleanedData = new ArrayList<>();
```

```
        for (String data : dataList) {
            //
            if (data != null && !data.trim().isEmpty()) {
                //
                if (!isNoiseString(data.trim())) {
                    cleanedData.add(data.trim());
                }
            }
        }
        return cleanedData;
    }
    /**
     *
     *
     * @param str
     * @return ，true；false
     */
    private boolean isNoiseString(String str) {
        for (String noise : NOISE_STRINGS) {
            if (str.equalsIgnoreCase(noise)) {
                return true;
            }
        }
        return false;
    }
    //
    public static void main(String[] args) {
        DataCleaningService service = new DataCleaningService();
        List<String> rawData = new ArrayList<>();
        rawData.add("validData1");
        rawData.add("    "); //
        rawData.add("noise1"); //
        rawData.add("validData2");
        rawData.add(null); //
        rawData.add("noise2"); //
        rawData.add(" validData3 "); //
        List<String> cleanedData = service.cleanAndDenoiseData(rawData);
        //
        for (String data : cleanedData) {
            System.out.println(data);
        }
    }
}
// Prediction.java
import javax.persistence.*;
@Entity
@Table(name = "predictions")
public class Prediction {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(name = "model_id")
    private Long modelId;
```

```
    @Column(name = "target_variable")
    private String targetVariable;
    @Column(name = "predicted_value")
    private Double predictedValue;
    @Column(name = "prediction_date")
    private LocalDateTime predictionDate;
    // gettersetter...
}
// PredictionRepository.java
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;
import java.time.LocalDateTime;
import java.util.List;
@Repository
public interface PredictionRepository extends JpaRepository<Prediction, Long> {
    // ID
    List<Prediction> findByModelId(Long modelId);
    //
    List<Prediction> findByTargetVariable(String targetVariable);
    //
    @Query("SELECT p FROM Prediction p WHERE p.predictionDate >= :afterDate
ORDER BY p.predictionDate ASC")
    List<Prediction> findPredictionsAfterDate(LocalDateTime afterDate);
    //
    Page<Prediction> findAll(Pageable pageable);
    //
    @Query("SELECT p FROM Prediction p WHERE p.predictionDate BETWEEN :startDate
AND :endDate ORDER BY p.predictionDate ASC")
    List<Prediction> findPredictionsByDateRange(LocalDateTime startDate, LocalDateTime
endDate);
    // ...
    //  ,
    //  ：Spring Data JPA@Query
}
import javax.persistence.*;
import java.util.List;
//
@Entity
@Table(name = "model_parameters")
public class ModelParameters {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "learning_rate")
    private double learningRate;

    @Column(name = "batch_size")
    private int batchSize;
```

```
    // gettersetter...
}
//
@Entity
@Table(name = "training_data")
public class TrainingData {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "input_data")
    private byte[] inputData; //

    @Column(name = "target_output")
    private byte[] targetOutput; //

    // gettersetter...
}
// （Repository）
public interface ModelParametersRepository extends JpaRepository<ModelParameters, Long> {
    // ，
}
public interface TrainingDataRepository extends JpaRepository<TrainingData, Long> {
    //
    List<TrainingData> findByInputData(byte[] inputData);

    // ...
}
// （Service），Repository
@Service
public class TrainingDataService {

    @Autowired
    private ModelParametersRepository modelParametersRepository;

    @Autowired
    private TrainingDataRepository trainingDataRepository;

    // ：
    public ModelParameters getModelParameters() {
        // （）
        return modelParametersRepository.findById(1L).orElse(null); // Optional
    }

    public List<TrainingData> getTrainingData() {
        // （）
        return trainingDataRepository.findAll();
    }

    // ：
    public void updateModelParameters(ModelParameters newParameters) {
```

```
            ModelParameters existingParameters = getModelParameters();
            if (existingParameters != null) {
                // （）
                existingParameters.setLearningRate(newParameters.getLearningRate());
                existingParameters.setBatchSize(newParameters.getBatchSize());
                modelParametersRepository.save(existingParameters);
            } else {
                // ，（）
            }
        }

        // ...
}
import java.util.List;
//
class DataPoint {
        private double value;
        private long timestamp;
        // 、gettersetter（）
        public DataPoint(double value, long timestamp) {
            this.value = value;
            this.timestamp = timestamp;
        }
        public double getValue() {
            return value;
        }
        public long getTimestamp() {
            return timestamp;
        }
}
//
class AnomalyDetectionService {
        //
        private static final double ANOMALY_THRESHOLD = 100.0;
        //
        public void detectAndAlertAnomalies(List<DataPoint> dataPoints) {
            for (DataPoint dataPoint : dataPoints) {
                double value = dataPoint.getValue();
                if (isAnomaly(value)) {
                    String message = "Anomaly detected! Value: " + value + " at timestamp: " +
dataPoint.getTimestamp();
                    alert(message);
                }
            }
        }
        //
        private boolean isAnomaly(double value) {
            return Math.abs(value) > ANOMALY_THRESHOLD;
        }
        //  （，、）
        private void alert(String message) {
            System.out.println("ALERT: " + message);
            // TODO: ，、
```

```
    }
    //  ,
    public static void main(String[] args) {
        AnomalyDetectionService service = new AnomalyDetectionService();
        List<DataPoint> dataPoints = List.of(
                new DataPoint(90.0, System.currentTimeMillis()),
                new DataPoint(120.0, System.currentTimeMillis() - 1000),
                new DataPoint(-110.0, System.currentTimeMillis() - 2000),
                new DataPoint(80.0, System.currentTimeMillis() - 3000)
        );
        service.detectAndAlertAnomalies(dataPoints);
    }
}
`alert`，。，、，。
`main`，。`AnomalyDetectionService`，`detectAndAlertAnomalies`。
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
public class RealtimeDataMonitor {
    //  ,
    private final BlockingQueue<String> dataQueue = new LinkedBlockingQueue<>();
    //
    private Thread producerThread;
    //
    private Thread consumerThread;
    //
    public void startMonitoring() {
        //
        producerThread = new Thread(() -> {
            while (true) {
                try {
                    //  （、）
                    String data = simulateRealtimeData();
                    dataQueue.put(data); //
                    Thread.sleep(1000); //
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    break;
                }
            }
        });
        producerThread.start();
        //
        consumerThread = new Thread(() -> {
            while (true) {
                try {
                    String data = dataQueue.take(); //
                    processData(data); //
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    break;
                }
            }
        });
```

```
            consumerThread.start();
        }
        //
        private String simulateRealtimeData() {
            //  , 、、
            return "Realtime Data: " + System.currentTimeMillis();
        }
        //
        private void processData(String data) {
            //
            //  : 、、
            System.out.println("Processing data: " + data);
        }
        //
        public void stopMonitoring() {
            producerThread.interrupt();
            consumerThread.interrupt();
            try {
                producerThread.join();
                consumerThread.join();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
        //  ,
        public static void main(String[] args) {
            RealtimeDataMonitor monitor = new RealtimeDataMonitor();
            monitor.startMonitoring(); //
            //
            try {
                Thread.sleep(5000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            monitor.stopMonitoring(); //
        }
}
import java.util.ArrayList;
import java.util.List;
public class LightSourceAndScintillatorBL {
        //
        static class LightSourceData {
            private double intensity; //
            private long timestamp; //
            //  、gettersetter...
            public LightSourceData(double intensity, long timestamp) {
                this.intensity = intensity;
                this.timestamp = timestamp;
            }
        }
        //
        static class ScintillatorResponse {
            private double responseLevel; //
```

```
        private long timestamp; //
        // 、gettersetter...
        public ScintillatorResponse(double responseLevel, long timestamp) {
            this.responseLevel = responseLevel;
            this.timestamp = timestamp;
        }
    }
    //
    public List<ScintillatorResponse> calculateScintillatorResponse(List<LightSourceData>
lightSourceDataList) {
        List<ScintillatorResponse> responses = new ArrayList<>();
        // ： 10%
        for (LightSourceData data : lightSourceDataList) {
            double responseLevel = data.getIntensity() * 0.1; // 10%
            long timestamp = data.getTimestamp();
            responses.add(new ScintillatorResponse(responseLevel, timestamp));
        }
        // ,、
        return responses;
    }
    // ,（,）
    public static void main(String[] args) {
        LightSourceAndScintillatorBL bl = new LightSourceAndScintillatorBL();
        List<LightSourceData> lightDataList = new ArrayList<>();
        lightDataList.add(new LightSourceData(100, System.currentTimeMillis()));
        lightDataList.add(new LightSourceData(200, System.currentTimeMillis() + 1000));
        lightDataList.add(new LightSourceData(50, System.currentTimeMillis() + 2000));
        List<ScintillatorResponse> responses =
bl.calculateScintillatorResponse(lightDataList);
        // （）
        for (ScintillatorResponse response : responses) {
            System.out.println("Response Level: " + response.getResponseLevel() + ",
Timestamp: " + response.getTimestamp());
        }
    }
}
import java.util.HashMap;
import java.util.Map;
public class PerformanceAwarePersistenceLayer {
    private static final Map<String, String> DATA_STORAGE = new HashMap<>();
    public void insertData(String key, String value) {
        long startTime = System.currentTimeMillis();
        // （Map）
        DATA_STORAGE.put(key, value);
        long endTime = System.currentTimeMillis();
        long elapsedTime = endTime - startTime;
        // ,
        // ,
        // System.out.println("Insert operation took: " + elapsedTime + "ms");
    }
    public String getData(String key) {
        long startTime = System.currentTimeMillis();
```

```
            //  （Map）
            String value = DATA_STORAGE.get(key);
            long endTime = System.currentTimeMillis();
            long elapsedTime = endTime - startTime;
            //  ,
            //  ,
            // System.out.println("Get operation took: " + elapsedTime + "ms");
            return value;
        }
        public void updateData(String key, String newValue) {
            long startTime = System.currentTimeMillis();
            //  （）
            if (DATA_STORAGE.containsKey(key)) {
                DATA_STORAGE.remove(key);
                DATA_STORAGE.put(key, newValue);
            }
            long endTime = System.currentTimeMillis();
            long elapsedTime = endTime - startTime;
            //  ,
            //  ,
            // System.out.println("Update operation took: " + elapsedTime + "ms");
        }
        public void deleteData(String key) {
            long startTime = System.currentTimeMillis();
            //  （Map）
            DATA_STORAGE.remove(key);
            long endTime = System.currentTimeMillis();
            long elapsedTime = endTime - startTime;
            //  ,
            //  ,
            // System.out.println("Delete operation took: " + elapsedTime + "ms");
        }
        //  , 、,
}
import java.util.HashMap;
import java.util.Map;
public class EnvironmentController {
        // EnvironmentService
        private EnvironmentService environmentService;
        public EnvironmentController(EnvironmentService environmentService) {
            this.environmentService = environmentService;
        }
        // HTTP（）
        public Map<String, Object> predictImpact(double temperature, double humidity) {
            //
            if (temperature < 0 || temperature > 100) {
                throw new IllegalArgumentException("Temperature must be between 0 and 100
degrees Celsius.");
            }
            if (humidity < 0 || humidity > 100) {
                throw new IllegalArgumentException("Humidity must be between 0 and 100
percent.");
```

```
        }
        //
        Map<String, Object> predictionResult = environmentService.predict(temperature,
humidity);
        // ,、
        // ,
        predictionResult.put("comfortLevel", "Moderate");
        predictionResult.put("healthImpact", "No significant impact");
        //
        if (temperature > 30 && humidity > 60) {
            predictionResult.put("heatStressWarning", "High risk of heat stress");
        }
        //
        return predictionResult;
    }
    // ,
    interface EnvironmentService {
        Map<String, Object> predict(double temperature, double humidity);
    }
    // ,
    static class SimpleEnvironmentService implements EnvironmentService {
        @Override
        public Map<String, Object> predict(double temperature, double humidity) {
            // ,
            Map<String, Object> result = new HashMap<>();
            // ,
            return result;
        }
    }
    //
    public static void main(String[] args) {
        //
        EnvironmentService service = new SimpleEnvironmentService();
        //
        EnvironmentController controller = new EnvironmentController(service);
        try {
            //
            Map<String, Object> result = controller.predictImpact(25, 50);
            System.out.println(result);
        } catch (IllegalArgumentException e) {
            System.err.println("Invalid input: " + e.getMessage());
        }
    }
}
import java.sql.*;
public class UserPersistenceLayer {
    private static final String JDBC_URL = "jdbc:mysql://localhost:3306/your_database";
    private static final String USER = "username";
    private static final String PASS = "password";
    //
    private Connection getConnection() throws SQLException {
        return DriverManager.getConnection(JDBC_URL, USER, PASS);
```

```
    }
    //  （）
    public void addUser(String username, String password) throws SQLException {
        try (Connection conn = getConnection();
            PreparedStatement stmt = conn.prepareStatement("INSERT INTO users (username, password) VALUES (?, ?)")) {
            stmt.setString(1, username);
            stmt.setString(2, password); //  ：，
            stmt.executeUpdate();
        }
    }
    //  （ID）
    public void updateUserPassword(int userId, String newPassword) throws SQLException {
        try (Connection conn = getConnection();
            PreparedStatement stmt = conn.prepareStatement("UPDATE users SET password = ? WHERE id = ?")) {
            stmt.setString(1, newPassword); //  ：，
            stmt.setInt(2, userId);
            stmt.executeUpdate();
        }
    }
    //  （）
    public User getUserById(int userId) throws SQLException {
        User user = null;
        try (Connection conn = getConnection();
            PreparedStatement stmt = conn.prepareStatement("SELECT username, password FROM users WHERE id = ?")) {
            stmt.setInt(1, userId);
            try (ResultSet rs = stmt.executeQuery()) {
                if (rs.next()) {
                    user = new User(rs.getInt("id"), rs.getString("username"), rs.getString("password"));
                }
            }
        }
        return user;
    }
    // User（）
    static class User {
        private int id;
        private String username;
        private String password; //  ：，String
        public User(int id, String username, String password) {
            this.id = id;
            this.username = username;
            this.password = password;
        }
        // GetterSetter（）
    }
    // ...  ，、
}
```

1. ****：。
2. ****：，。
3. ****：，。，SQL。
4. ****：，。
5. ****：，。

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.Date;
//
class LogRecord {
    private String level;
    private String message;
    private Date timestamp;
    // 、gettersetter
}
// DAO
public class MonitoringDao {
    private static final String JDBC_URL = "jdbc:mysql://localhost:3306/monitoring_db";
    private static final String USER = "username";
    private static final String PASSWORD = "password";
    // Logger，
    private static final Logger LOGGER = LoggerFactory.getLogger(MonitoringDao.class);
    //
    public void insertLogRecord(LogRecord logRecord) {
        String sql = "INSERT INTO log_records (level, message, timestamp) VALUES
(?, ?, ?)";
        try (Connection connection = DriverManager.getConnection(JDBC_URL, USER,
PASSWORD);
                PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, logRecord.getLevel());
            statement.setString(2, logRecord.getMessage());
            statement.setTimestamp(3,                                    new
java.sql.Timestamp(logRecord.getTimestamp().getTime()));
            int rowsAffected = statement.executeUpdate();
            if (rowsAffected > 0) {
                LOGGER.info("Log record inserted successfully.");
            } else {
                LOGGER.warn("No rows were affected by the insert operation.");
            }
        } catch (SQLException e) {
            LOGGER.error("Error inserting log record: ", e);
        }
    }
    //  （）
    public String getSystemInfo() {
        //  , ,
        return "System is running normally.";
    }
    // LoggerLoggerFactory（）
    // private static final Logger LOGGER = LoggerFactory.getLogger(MonitoringDao.class);
```

```
    // （）
    public static void main(String[] args) {
        MonitoringDao dao = new MonitoringDao();
        LogRecord logRecord = new LogRecord();
        logRecord.setLevel("INFO");
        logRecord.setMessage("System started successfully.");
        logRecord.setTimestamp(new Date());
        dao.insertLogRecord(logRecord);
        System.out.println(dao.getSystemInfo());
    }
}
```

****：

1. `log_records`，（`level`, `message`, `timestamp`）。
2. `Logger``LoggerFactory`，，SLF4JLog4j。
3. ，DAO，。
4. `getSystemInfo`，。，。

```
import java.util.HashMap;
import java.util.Map;
public class UserInteractionService {
    // ，
    private Map<String, String> userData = new HashMap<>();
    //
    public boolean registerUser(String username, String password) {
        if (username == null || password == null || username.isEmpty() || password.isEmpty())
{
            return false; // ，
        }
        if (userData.containsKey(username)) {
            return false; // ，
        }
        userData.put(username, password); //
        return true; //
    }
    //
    public boolean loginUser(String username, String password) {
        if (username == null || password == null || username.isEmpty() || password.isEmpty())
{
            return false; // ，
        }
        String storedPassword = userData.get(username);
        return storedPassword != null && storedPassword.equals(password); //
    }
    //
    public boolean updateUserInfo(String username, String newInfo) {
        if (username == null || newInfo == null || username.isEmpty() || newInfo.isEmpty()) {
            return false; // ，
        }
        // ，（）
        userData.put(username + "_info", newInfo);
        return true; //
    }
    //
```

```
    public String getUserInfo(String username) {
        if (username == null || username.isEmpty()) {
            return null; //  ，
        }
        return userData.getOrDefault(username + "_info", "No info available"); //
    }
    //  （，）
    public boolean logoutUser(String username) {
        if (username == null || username.isEmpty()) {
            return false; //  ，
        }
        userData.remove(username);
        userData.remove(username + "_info");
        return true; //
    }
    // ...
}
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.Map;
//
interface PredictionService {
    double predict(double[] input, String modelName);
}
// PredictionService
// ...（）
//  ，
@RestController
@RequestMapping("/predictions")
public class PredictionController {
    // Map
    private final Map<String, PredictionService> modelMap;
    @Autowired
    public PredictionController(List<PredictionService> predictionServices) {
        //  ，
        modelMap = predictionServices.stream()
                .collect(Map.ofEntries(predictionServices.stream()
                        .map(service  ->  Map.entry(service.getClass().getSimpleName(),
service))
                        .toList()));
    }
    // POST，
    @PostMapping("/ensemble")
    public  ResponseEntity<Double>  ensemblePrediction(@RequestBody  Map<String,
Object> requestData) {
        try {
            //
            double[] inputFeatures = (double[]) requestData.get("inputFeatures");
            List<String> modelNames = (List<String>) requestData.get("modelNames");
```

```
                if (inputFeatures == null || modelNames == null || modelNames.isEmpty()) {
                    return new ResponseEntity<>("Input features or model names are missing
or empty", HttpStatus.BAD_REQUEST);
                }
                //  （）
                double ensemblePrediction = 0.0;
                double count = 0.0;
                for (String modelName : modelNames) {
                    //
                    if (!modelMap.containsKey(modelName)) {
                        continue; //
                    }
                    //
                    double   prediction   =   modelMap.get(modelName).predict(inputFeatures,
modelName);
                    ensemblePrediction += prediction;
                    count++;
                }
                //
                if (count > 0) {
                    ensemblePrediction /= count;
                }
                //
                return new ResponseEntity<>(ensemblePrediction, HttpStatus.OK);
            } catch (Exception e) {
                //
                e.printStackTrace();
                return   new   ResponseEntity<>("An   error   occurred   while   predicting",
HttpStatus.INTERNAL_SERVER_ERROR);
            }
        }
}
`ensemblePrediction`POST，。，。，，。
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/lifespan")
public class LifespanPredictionController {
    @Autowired
    private ServiceLayer serviceLayer;
    @PostMapping("/predict")
    public  ResponseEntity<LifespanPredictionResponse>  predictLifespan(@RequestBody
LifespanPredictionRequest request) {
        try {
            LifespanPredictionResult                    result                    =
serviceLayer.predictLifespan(request.getDeviceData());
            LifespanPredictionResponse response = new LifespanPredictionResponse();
            response.setEstimatedLifespan(result.getEstimatedLifespan());
            response.setMaintenanceAdvice(result.getMaintenanceAdvice());
            return new ResponseEntity<>(response, HttpStatus.OK);
        } catch (Exception e) {
```

```
            //  ,
            return new ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }
    //
    static class LifespanPredictionRequest {
        private DeviceData deviceData;
        // Getter    Setter
        public DeviceData getDeviceData() {
            return deviceData;
        }
        public void setDeviceData(DeviceData deviceData) {
            this.deviceData = deviceData;
        }
    }
    static class DeviceData {
        //  , 、、
        //
        private String model;
        private int usageHours;
        // ...    Getter/Setter
        // Getter    Setter
        public String getModel() {
            return model;
        }
        public void setModel(String model) {
            this.model = model;
        }
        public int getUsageHours() {
            return usageHours;
        }
        public void setUsageHours(int usageHours) {
            this.usageHours = usageHours;
        }
        // ...    Getter/Setter
    }
    static class LifespanPredictionResult {
        private int estimatedLifespan;
        private String maintenanceAdvice;
        // Getter    Setter
        public int getEstimatedLifespan() {
            return estimatedLifespan;
        }
        public void setEstimatedLifespan(int estimatedLifespan) {
            this.estimatedLifespan = estimatedLifespan;
        }
        public String getMaintenanceAdvice() {
            return maintenanceAdvice;
        }
        public void setMaintenanceAdvice(String maintenanceAdvice) {
            this.maintenanceAdvice = maintenanceAdvice;
        }
    }
```

```
    static class LifespanPredictionResponse {
        private int estimatedLifespan;
        private String maintenanceAdvice;
        // Getter   Setter
        public int getEstimatedLifespan() {
            return estimatedLifespan;
        }
        public void setEstimatedLifespan(int estimatedLifespan) {
            this.estimatedLifespan = estimatedLifespan;
        }
        public String getMaintenanceAdvice() {
            return maintenanceAdvice;
        }
        public void setMaintenanceAdvice(String maintenanceAdvice) {
            this.maintenanceAdvice = maintenanceAdvice;
        }
    }
}
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.logging.Level;
import java.util.logging.Logger;
@RestController
@RequestMapping("/monitor")
public class MonitoringController {
    private static final Logger logger =
Logger.getLogger(MonitoringController.class.getName());
    //
    private Map<String, Object> systemStatus = new HashMap<>();
    //
    public MonitoringController() {
        systemStatus.put("uptime", new Date().toString());
        systemStatus.put("memoryUsage", "50%");
        systemStatus.put("cpuUsage", "30%");
        // ...
    }
    //
    @GetMapping("/status")
    public Map<String, Object> getSystemStatus() {
        logger.log(Level.INFO, "Requesting system status.");
        return systemStatus;
    }
    //  （，）
    @PostMapping("/update")
    public String updateSystemStatus(@RequestBody Map<String, String> newStatus) {
        logger.log(Level.INFO, "Updating system status with new data: {0}", newStatus);
        for (Map.Entry<String, String> entry : newStatus.entrySet()) {
            systemStatus.put(entry.getKey(), entry.getValue());
        }
        logger.log(Level.INFO, "System status updated successfully.");
        return "System status updated";
    }
```

```
    // （，）
    @PostMapping("/log-event")
    public String logEvent(@RequestParam String eventName, @RequestParam(required = false) String details) {
        logger.log(Level.INFO, "Event logged: {0}. Details: {1}", new Object[]{eventName, details == null ? "N/A" : details});
        return "Event logged successfully";
    }
    // （，）
    @PostMapping("/trigger-alert")
    public String triggerAlert(@RequestParam String alertName, @RequestParam String alertMessage) {
        // ，、API
        logger.log(Level.SEVERE, "Alert triggered: {0}. Message: {1}", new Object[]{alertName, alertMessage});
        return "Alert triggered";
    }
    // API...
}
import javax.persistence.*;
@Entity
@Table(name = "model_a_predictions")
public class ModelAPrediction {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Double predictionValue;
    // ...
    // 、gettersetter...
}
@Entity
@Table(name = "model_b_predictions")
public class ModelBPrediction {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Double predictionValue;
    // ...
    // 、gettersetter...
}
@Entity
@Table(name = "integrated_predictions")
public class IntegratedPrediction {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @ManyToOne
    @JoinColumn(name = "model_a_prediction_id")
    private ModelAPrediction modelAPrediction;
    @ManyToOne
    @JoinColumn(name = "model_b_prediction_id")
    private ModelBPrediction modelBPrediction;
    private Double finalPredictionValue; //
```

```
    // ...
    // 、gettersetter...
}
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
@Repository
public interface ModelAPredictionRepository extends JpaRepository<ModelAPrediction,
Long> {
    // ...
}
@Repository
public interface ModelBPredictionRepository extends JpaRepository<ModelBPrediction,
Long> {
    // ...
}
@Repository
public interface IntegratedPredictionRepository extends JpaRepository<IntegratedPrediction,
Long> {
    // ...
    // ID
    IntegratedPrediction save(IntegratedPrediction integratedPrediction);
}
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
@Service
public class IntegratedPredictionService {
    @Autowired
    private ModelAPredictionRepository modelAPredictionRepository;
    @Autowired
    private ModelBPredictionRepository modelBPredictionRepository;
    @Autowired
    private IntegratedPredictionRepository integratedPredictionRepository;
    public IntegratedPrediction integratePredictions(ModelAPrediction modelAPrediction,
ModelBPrediction modelBPrediction) {
        //
        // ,
        Double finalPredictionValue = (modelAPrediction.getPredictionValue() +
modelBPrediction.getPredictionValue()) / 2;
        IntegratedPrediction integratedPrediction = new IntegratedPrediction();
        integratedPrediction.setModelAPrediction(modelAPrediction);
        integratedPrediction.setModelBPrediction(modelBPrediction);
        integratedPrediction.setFinalPredictionValue(finalPredictionValue);
        return integratedPredictionRepository.save(integratedPrediction);
    }
    // ...
}
import java.sql.*;
public class EnvironmentPersistence {
    private static final String JDBC_URL = "jdbc:mysql://localhost:3306/environment_data";
    private static final String USER = "username";
    private static final String PASSWORD = "password";
    // SQL（）
    private static final String CREATE_TABLE_SQL = "CREATE TABLE IF NOT EXISTS
```

```
environment_data (" +
                "id INT AUTO_INCREMENT PRIMARY KEY," +
                "temperature FLOAT NOT NULL," +
                "humidity FLOAT NOT NULL," +
                "prediction VARCHAR(255) NOT NULL" +
                ")";
    // SQL
    private static final String INSERT_DATA_SQL = "INSERT INTO environment_data
(temperature, humidity, prediction) VALUES (?, ?, ?)";
    // SQL（，）
    private static final String QUERY_DATA_SQL = "SELECT * FROM environment_data";
    //
    private Connection connectToDatabase() throws SQLException {
        return DriverManager.getConnection(JDBC_URL, USER, PASSWORD);
    }
    //  （）
    public void createTable() throws SQLException {
        try (Connection connection = connectToDatabase();
                Statement statement = connection.createStatement()) {
                statement.execute(CREATE_TABLE_SQL);
        }
    }
    //
    public void insertData(float temperature, float humidity, String prediction) throws
SQLException {
        try (Connection connection = connectToDatabase();
                PreparedStatement preparedStatement =
connection.prepareStatement(INSERT_DATA_SQL)) {
                preparedStatement.setFloat(1, temperature);
                preparedStatement.setFloat(2, humidity);
                preparedStatement.setString(3, prediction);
                preparedStatement.executeUpdate();
        }
    }
    //
    public ResultSet queryData() throws SQLException {
        try (Connection connection = connectToDatabase();
                Statement statement = connection.createStatement()) {
                return statement.executeQuery(QUERY_DATA_SQL);
        }
    }
    //  （，）
    public static void main(String[] args) {
        EnvironmentPersistence persistence = new EnvironmentPersistence();
        try {
            //  （，）
            persistence.createTable();
            //
            persistence.insertData(25.0f, 60.0f, "Hot and Humid");
            persistence.insertData(15.0f, 40.0f, "Cool and Dry");
            //  （）
            ResultSet resultSet = persistence.queryData();
            while (resultSet.next()) {
```

```
                    System.out.println("ID: " + resultSet.getInt("id") +
                                ", Temperature: " + resultSet.getFloat("temperature") +
                                ", Humidity: " + resultSet.getFloat("humidity") +
                                ", Prediction: " + resultSet.getString("prediction"));
                }
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
}
import java.sql.*;
public class FeatureSelectionDAO {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/feature_selection_db";
    private static final String USER = "root";
    private static final String PASS = "password";
    //
    public void saveFeatureSelection(int featureId, String featureName, boolean isSelected)
{
        String sql = "INSERT INTO feature_selection (feature_id, feature_name, is_selected)
VALUES (?, ?, ?)";
        try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, featureId);
            pstmt.setString(2, featureName);
            pstmt.setBoolean(3, isSelected);
            pstmt.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
    //
    public void updateFeatureSelection(int featureId, boolean isSelected) {
        String sql = "UPDATE feature_selection SET is_selected = ? WHERE feature_id
= ?";
        try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setBoolean(1, isSelected);
            pstmt.setInt(2, featureId);
            pstmt.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
    //
    public ResultSet retrieveAllFeatureSelections() {
        String sql = "SELECT * FROM feature_selection";
        try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            return rs;
        } catch (SQLException e) {
            e.printStackTrace();
            return null;
```

```
            }
        }
        // ID
        public ResultSet retrieveFeatureSelectionById(int featureId) {
            String sql = "SELECT * FROM feature_selection WHERE feature_id = ?";
            try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
                  PreparedStatement pstmt = conn.prepareStatement(sql)) {
                pstmt.setInt(1, featureId);
                return pstmt.executeQuery();
            } catch (SQLException e) {
                e.printStackTrace();
                return null;
            }
        }
        // ResultSetStatement（），。
        // ...
}
**User.java**
import javax.persistence.*;
import java.util.List;
@Entity
@Table(name = "users")
public class User {
        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Long id;
        @Column(name = "username", unique = true, nullable = false)
        private String username;
        @Column(name = "password", nullable = false)
        private String password; // ：
        @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval =
true)
        private List<UserPermission> userPermissions;
        // Getters, Setters, and other methods omitted for brevity
}
**Permission.java**
import javax.persistence.*;
import java.util.List;
@Entity
@Table(name = "permissions")
public class Permission {
        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Long id;
        @Column(name = "name", unique = true, nullable = false)
        private String name;
        @OneToMany(mappedBy = "permission", cascade = CascadeType.ALL,
orphanRemoval = true)
        private List<UserPermission> userPermissions;
        // Getters, Setters, and other methods omitted for brevity
}
**UserPermission.java**
import javax.persistence.*;
```

```
@Entity
@Table(name = "user_permissions")
public class UserPermission {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private User user;
    @ManyToOne
    @JoinColumn(name = "permission_id", nullable = false)
    private Permission permission;
    // Getters, Setters, and other methods omitted for brevity
}
**UserRepository.java**
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    //  ,
}
**PermissionRepository.java**
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
@Repository
public interface PermissionRepository extends JpaRepository<Permission, Long> {
    //  ,
}
**UserPermissionRepository.java**
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
@Repository
public interface UserPermissionRepository extends JpaRepository<UserPermission, Long> {
    //  ,
}
import javax.persistence.*;
@Entity
@Table(name = "abnormal_values")
public class AbnormalValue {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(name = "value", nullable = false)
    private Double value;
    @Column(name = "timestamp", nullable = false)
    private LocalDateTime timestamp;
    @Column(name = "alert_sent", nullable = false)
    private boolean alertSent = false;
    // Getters and Setters
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
```

```
        this.id = id;
    }
    public Double getValue() {
        return value;
    }
    public void setValue(Double value) {
        this.value = value;
    }
    public LocalDateTime getTimestamp() {
        return timestamp;
    }
    public void setTimestamp(LocalDateTime timestamp) {
        this.timestamp = timestamp;
    }
    public boolean isAlertSent() {
        return alertSent;
    }
    public void setAlertSent(boolean alertSent) {
        this.alertSent = alertSent;
    }
}
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.transaction.annotation.Transactional;
import java.time.LocalDateTime;
import java.util.List;
public interface AbnormalValueRepository extends JpaRepository<AbnormalValue, Long> {
    //
    @Query("SELECT a FROM AbnormalValue a WHERE a.alertSent = false AND
a.value > ?1 ORDER BY a.timestamp DESC")
    List<AbnormalValue> findUnsentAlertsByThreshold(Double threshold);
    //
    @Modifying
    @Transactional
    @Query("UPDATE AbnormalValue a SET a.alertSent = true WHERE a.id = ?1")
    void markAlertAsSent(Long id);
    //
    void save(AbnormalValue abnormalValue);
    //
    @Query("SELECT a FROM AbnormalValue a WHERE a.timestamp BETWEEN ?1
AND ?2 ORDER BY a.timestamp DESC")
    List<AbnormalValue> findByTimestampBetween(LocalDateTime start, LocalDateTime
end);
}
import javax.persistence.*;
@Entity
@Table(name = "models")
public class ModelEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(name = "name", nullable = false, unique = true)
```

```
    private String name;
    @Column(name = "initialized", nullable = false)
    private boolean initialized;
    // Getters and setters
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public boolean isInitialized() {
        return initialized;
    }
    public void setInitialized(boolean initialized) {
        this.initialized = initialized;
    }
}
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
@Repository
public interface ModelRepository extends JpaRepository<ModelEntity, Long> {
    // ：
    ModelEntity findByName(String name);
    // ：
    @Modifying
    @Query("UPDATE ModelEntity m SET m.initialized = true WHERE m.name = :name")
    int initializeModel(@Param("name") String name);
    // ：
    List<ModelEntity> findAllByInitializedIsTrue();
}
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;
@Service
public class ModelService {
    @Autowired
    private ModelRepository modelRepository;
    //
    public ModelEntity saveModel(ModelEntity model) {
        return modelRepository.save(model);
    }
    //
    public void initializeModel(String name) {
        modelRepository.initializeModel(name);
    }
    //
    public List<ModelEntity> findAllInitializedModels() {
```

```
        return modelRepository.findAllByInitializedIsTrue();
    }
}
import java.time.LocalDateTime;
public class SystemMonitor {

    //
    private final Logger logger;
    //  ,
    public SystemMonitor(Logger logger) {
        this.logger = logger;
    }
    //
    private void logInfo(String message) {
        logger.info("{}: {}", LocalDateTime.now(), message);
    }
    //  , ,
    public void monitorSystem() {
        // CPU（）
        double cpuUsage = Math.random() * 100; // 0-100
        //
        logInfo("CPU Usage: " + cpuUsage + "%");
        //  （）
        long freeMemory = Runtime.getRuntime().freeMemory();
        long totalMemory = Runtime.getRuntime().totalMemory();
        double memoryUsage = 100.0 * (totalMemory - freeMemory) / totalMemory;
        //
        logInfo("Memory Usage: " + memoryUsage + "%");
        //  , 、
        //  （、）
        long startTime = System.currentTimeMillis();
        try {
            //
            Thread.sleep(1000); // 1
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            logger.error("Monitoring task was interrupted", e);
            return;
        }
        long endTime = System.currentTimeMillis();
        //
        logInfo("Monitoring task completed in " + (endTime - startTime) + "ms");
    }
    // Logger，
    interface Logger {
        void info(String message);
        void error(String message, Throwable throwable);
    }
    //  ,
    public static void main(String[] args) {
        // Logger，
        Logger logger = new Logger() {
```

```
                @Override
                public void info(String message) {
                    System.out.println("[INFO] " + message);
                }
                @Override
                public void error(String message, Throwable throwable) {
                    System.err.println("[ERROR] " + message);
                    throwable.printStackTrace();
                }
            };
            // SystemMonitor
            SystemMonitor monitor = new SystemMonitor(logger);
            monitor.monitorSystem();
        }
}
import java.io.IOException;
import java.util.List;
import javax.servlet.http.HttpServletResponse;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
@Controller
@RequestMapping("/predictions")
public class PredictionController {
    @Autowired
    private PredictionService predictionService;
    @Autowired
    private ReportGenerator reportGenerator;
    @GetMapping("/export")
    public void exportPredictions(HttpServletResponse response) throws IOException {
        //
        List<PredictionResult> predictionResults = predictionService.getLatestPredictions();
        // ，
        response.setContentType("application/vnd.ms-excel");
        response.setHeader("Content-Disposition", "attachment; filename=predictions.xls");
        // Excel
        reportGenerator.generateExcelReport(predictionResults,
response.getOutputStream());
        //
        response.flushBuffer();
        response.getOutputStream().close();
    }
    @GetMapping("/report")
    public String generateReport(HttpServletResponse response) {
        //
        List<PredictionResult> predictionResults = predictionService.getLatestPredictions();
        // HTML（，）
        String reportHtml = reportGenerator.generateHtmlReport(predictionResults);
        // ，HTML、
        // ，
        return "Report has been generated: " + reportHtml.substring(0,
Math.min(reportHtml.length(), 50)) + "...";
```

```
        // ：，，、、
    }
    // PredictionResult（，）
    static class PredictionResult {
        //
        // ...
    }
    // PredictionService（，）
    interface PredictionService {
        List<PredictionResult> getLatestPredictions();
        // ...
    }
    // ReportGenerator（，）
    static class ReportGenerator {
        // Excel
        void generateExcelReport(List<PredictionResult> results, java.io.OutputStream
outputStream) throws IOException {
            // Excel...
        }
        // HTML
        String generateHtmlReport(List<PredictionResult> results) {
            // HTML...
            return "HTML Report Content"; //
        }
    }
}
public class RawData {
    private Long id;
    private String noisyColumn; //
    // ...
    // GetterSetter...
}
import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Param;
import org.apache.ibatis.annotations.Select;
import java.util.List;
@Mapper
public interface RawDataMapper {
    // ，XML
    @Select("SELECT * FROM RawData")
    List<RawData> selectAllWithCleaning();
    // ...
}
xml
<mapper namespace="com.example.mapper.RawDataMapper">
    <!-- ， -->
    <resultMap id="rawDataResultMap" type="com.example.model.RawData">
        <id property="id" column="id"/>
        <result property="noisyColumn" column="noisy_column"
typeHandler="com.example.typehandler.CleanedStringTypeHandler"/>
        <!-- ... -->
    </resultMap>
```

```
    <!--    -->
    <select id="selectAllWithCleaning" resultMap="rawDataResultMap">
        SELECT * FROM RawData
    </select>
    <!-- ... -->
</mapper>
import org.apache.ibatis.type.BaseTypeHandler;
import org.apache.ibatis.type.JdbcType;
import java.sql.CallableStatement;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
public class CleanedStringTypeHandler extends BaseTypeHandler<String> {
    @Override
    public void setNonNullParameter(PreparedStatement ps, int i, String parameter,
JdbcType jdbcType) throws SQLException {
        //  ,
        super.setNonNullParameter(ps, i, parameter, jdbcType);
    }
    @Override
    public String getNullableResult(ResultSet rs, String columnName) throws SQLException
{
        String originalValue = rs.getString(columnName);
        //  ,
        String cleanedValue = originalValue.replaceAll("[^a-zA-Z0-9]", ""); //
        return cleanedValue;
    }
    // ...
}
import java.util.Arrays;
public class ModelTrainingAndOptimizationService {
    //  ,
    private static class SimpleModel {
        private double[] weights;
        public SimpleModel(int inputSize) {
            this.weights = new double[inputSize];
            Arrays.fill(this.weights, 0.01); //
        }
        //
        public double forward(double[] input) {
            //
            double output = 0;
            for (int i = 0; i < input.length; i++) {
                output += input[i] * weights[i];
            }
            return output;
        }
        //  （）
        public void updateWeights(double[] gradients, double learningRate) {
            for (int i = 0; i < weights.length; i++) {
                weights[i] -= learningRate * gradients[i];
            }
        }
```

```
    }
    //
    private static final double[][] TRAIN_DATA_X = {
        {1, 2},
        {2, 3},
        {3, 4}
    };
    private static final double[] TRAIN_DATA_Y = {2, 3, 4};
    //
    private static double calculateLoss(SimpleModel model, double[] input, double target) {
        //
        double prediction = model.forward(input);
        return Math.pow(prediction - target, 2);
    }
    //
    private static double[] calculateGradients(SimpleModel model, double[] input, double
target) {
        // ，
        double prediction = model.forward(input);
        double gradient = 2 * (prediction - target) * input[0]; //
        return new double[]{gradient}; // ，
    }
    //
    public SimpleModel trainModel(int epochs, double learningRate) {
        SimpleModel model = new SimpleModel(TRAIN_DATA_X[0].length);
        for (int epoch = 0; epoch < epochs; epoch++) {
            for (int i = 0; i < TRAIN_DATA_X.length; i++) {
                double[] gradients = calculateGradients(model, TRAIN_DATA_X[i],
TRAIN_DATA_Y[i]);
                model.updateWeights(gradients, learningRate);
            }
            //
        }
        return model;
    }
    // ，
    public static void main(String[] args) {
        ModelTrainingAndOptimizationService service = new
ModelTrainingAndOptimizationService();
        SimpleModel trainedModel = service.trainModel(1000, 0.01); // 1000，0.01
        // trainedModel
    }
}
import java.util.HashMap;
import java.util.Map;
//
public class User {
    private String id;
    private String name;
    private int age;
    // 、gettersetter，
    // ...
    @Override
```

```
    public String toString() {
        return "User{" +
                "id='" + id + '\'' +
                ", name='" + name + '\'' +
                ", age=" + age +
                '}';
    }
}
//
public class UserService {
    //  ,
    private Map<String, User> users = new HashMap<>();
    //
    public User initializeUser(String name, int age) {
        // ID（,）
        String id = "user-" + System.currentTimeMillis();
        //
        User user = new User();
        user.setId(id);
        user.setName(name);
        user.setAge(age);
        //  （）
        users.put(id, user);
        //
        return user;
    }
    //  （,）
    public User getUserById(String id) {
        return users.get(id);
    }
    public void updateUser(String id, String newName, int newAge) {
        User user = users.get(id);
        if (user != null) {
            user.setName(newName);
            user.setAge(newAge);
        }
    }
    public void deleteUser(String id) {
        users.remove(id);
    }
    public static void main(String[] args) {
        UserService userService = new UserService();
        //
        User user = userService.initializeUser("John Doe", 30);
        System.out.println("Initialized user: " + user);
        //  （）
        User retrievedUser = userService.getUserById(user.getId());
        System.out.println("Retrieved user: " + retrievedUser);
        //  （）
        userService.updateUser(user.getId(), "John Smith", 31);
        retrievedUser = userService.getUserById(user.getId());
        System.out.println("Updated user: " + retrievedUser);
```

```
            //  （）
            userService.deleteUser(user.getId());
            retrievedUser = userService.getUserById(user.getId());
            System.out.println("User after deletion: " + (retrievedUser != null ? retrievedUser : "null"));
        }
}
import java.util.ArrayList;
import java.util.List;
public class DataCollectionAndPreprocessingService {
        // ， List<String>
        private static List<String> dataSource = new ArrayList<>();
        static {
            //  ，
            dataSource.add("123");
            dataSource.add("456.78");
            dataSource.add(""); //
            dataSource.add("abc"); //
            dataSource.add("910.11");
        }
        //  ：
        public List<String> collectData() {
            return new ArrayList<>(dataSource);
        }
        //  ：，、
        public List<Double> preprocessData(List<String> rawData) {
            List<Double> processedData = new ArrayList<>();
            for (String data : rawData) {
                //
                if (data != null && !data.isEmpty() && isNumeric(data)) {
                    try {
                        //  （Double）
                        processedData.add(Double.parseDouble(data));
                    } catch (NumberFormatException e) {

                    }
                }
            }
            return processedData;
        }
        private boolean isNumeric(String str) {
            if (str == null) {
                return false;
            }
            try {
                Double.parseDouble(str);
            } catch (NumberFormatException nfe) {
                return false;
            }
            return true;
        }
}
```